

Statistics 6910: Data Frames, Lists, Matrices

AND the Apply Family

Reminder: Data Frame

- *Ordered* container of vectors
- Vectors must all be the *same length*
- Vectors can be *different types*

Wireless Data

- There are 5 wireless access points in a building
- A laptop emits a signal and the strength of the signal (S1, S2, ..., S5) is recorded at each access point.
- Also recorded is the location (x & y) of the laptop in the building.
- Measurements are taken at 254 locations.

```
w =  
read.table("http://www.math.usu.edu/~symanzik/teaching/  
2013_stat6910_003_fall/Files/wireless.txt", header = TRUE)
```

Helpful Functions for finding information out about the data frame

class(w)

```
[1] "data.frame"
```

names(w)

```
[1] "x" "y" "S1" "S2" "S3" "S4" "S5"
```

dim(w)

```
[1] 259 7
```

head(w)

```

  x  y S1 S2 S3 S4 S5
1 225.0 144 -92 -78 -49 -92 -92
2   0.0 144 -75 -92 -87 -47 -92
3 111.0 132 -92 -92 -80 -70 -65
4 110.7 132 -87 -92 -67 -67 -62
5 105.0 132 -92 -92 -79 -66 -61
6   99.0 132 -92 -92 -79 -70 -61

```

summary(w\$S1)

```

Min. 1st Qu.  Median    Mean 3rd Qu.   Max.
-92.00 -90.00 -80.00 -77.41 -71.00 -33.00

```

w[w\$x > 200, "S2"]

```

[1] -78 -74 -35 -43 -46 -43 -41 -48 -68 -71 -67 -70 -58 -61
[15] -64 -73 -71 -67 -68 -61 -60 -57 -58 -56 -48 -51 -68 -54
[29] -51 -48 -46 -35 -67

```

We subset rows and columns of data frames

We subset by **position**, **exclusion**, **logical**, **name**, and **all**

Lists

Reminder: Lists

- Data frames are actually a special kind of *list*.
- Unlike a data frame each element in a list can have a different length.
- Actually, each element can be either a list, data frame, vector, matrix ...
- Note that the elements are not associated with one another by position, as they were in a given row of a data frame.

Rainfall

- Daily rainfall collected for 5 weather stations in Colorado
- **rain** is a list of length 5
 - One element for each station
 - Each element is a numeric vector of rain measurements
 - Stations not in operation for the same length of time
- **day** has the same structure as rain

```
load(url("http://www.math.usu.edu/~symanzik/teaching/2013_stat6910_003_fall/Files/rainfallCO.rda"))
```

```
class(rain)
```

```
[1] "list"
```

```
length(rain)
```

```
[1] 5
```

```
names(rain)
```

```
[1] "st050183" "st050263" "st050712" "st050843" "st050945"
```

Indexing lists

- Lists can be indexed by name, using \$.

```
class(rain$st050183)
```

```
[1] "numeric"
```

```
length(rain$st050183)
```

```
[1] 9878
```

```
head(rain$st050183)
```

```
[1] 0 10 11 1 0 0
```

Indexing lists

- Lists can also be indexed like vectors, using []. The result will be another list.

```
class(rain["st050183"])
```

```
[1] "list"
```

```
length(rain["st050183"])
```

```
[1] 1
```

Indexing lists

- To extract individual elements of a list, enclose the index in `[[ ]]`. The result will be an object of the same type as the element of the list.

```
class(rain[[1]])
```

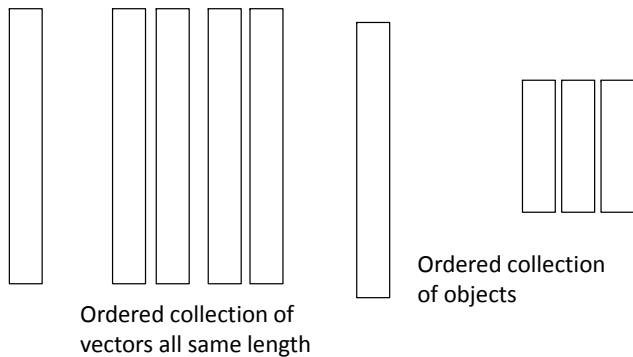
```
[1] "numeric"
```

```
head(rain[[1]])
```

```
[1] 0 10 11 1 0 0
```

Reminder: Summary of Data Structures

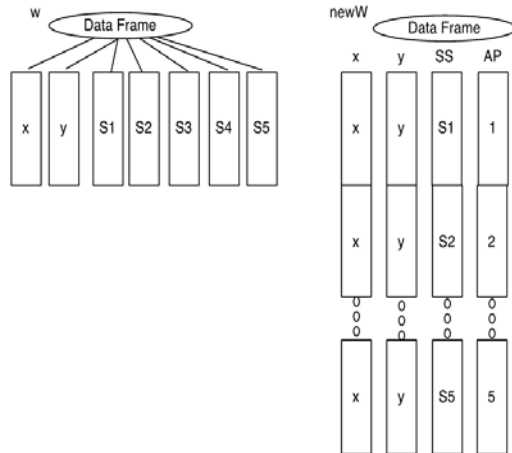
Vector – Data frame - List



Return to the Wifi problem

We will reformat the data frame into another data frame

Revise the Structure



```
X = rep(w$x, 5)
Y = rep(w$y, 5)
AP = rep(1:5, each = nrow(w))
SS = c(w$S1, w$S2, w$S3, w$S4, w$S5)
```

```
newW = data.frame(x = X, y = Y, AP, SS)
```

```
head(newW)
```

```
x y AP SS
1 225.0 144 1 -92
2 0.0 144 1 -75
3 111.0 132 1 -92
4 110.7 132 1 -87
5 105.0 132 1 -92
6 99.0 132 1 -92
```

Apply Functions

- Sometimes we want an operation to be applied to each element of a list, to each vector in a data frame, or to individual dimensions of a matrix
- R provides the *apply* mechanism to do this.
- There are several apply functions:
 - `sapply()` and `lapply()` for lists and data frames
 - `apply()` for matrices
 - `tapply()` for “tables”, i.e. ragged arrays as vectors

- With these functions we can avoid looping, and instead we write code that is meaningful in a statistical setting.
- For example with our list of rainfall data, each element represents the measurements taken at a particular weather station and when we think about studying the average rainfall at each station - we don't think in terms of loops.

lapply() and sapply()

- The **lapply** and **sapply** both apply a specified function to each element of a list.
- The former returns a list object and the latter a vector when possible.

Mean rainfall at each station

lapply(rain, mean)

```
$st050183
[1] 6.631707

$st050263
[1] 3.798993

$st050712
[1] 5.102299

$st050843
[1] 6.084607

$st050945
[1] 4.549296
```

sapply(rain, mean)

```
st050183 st050263 st050712
6.631707 3.798993 5.102299

st050843 st050945
6.084607 4.549296
```

Additional arguments

**lapply(rain, mean, na.rm = TRUE,
trim = 0.1)**

```
$st050183
[1] 2.393978

$st050263
[1] 0.9875949

$st050712
[1] 0.7895235

$st050843
[1] 1.238481

$st050945
[1] 0.7366283
```

args(lapply)
function (X, FUN, ...)

X takes the list object
FUN is the function to
apply to each element
in X

... allows any number
of arguments to be
passed to FUN

tapply()

- This function is useful to apply a function to subsets of a vector.

`x = 1:10`

`[1] 1 2 3 4 5 6 7 8 9 10`

`v = c(1, 1, 1, 0, 0, 0, 1, 1, 1, 0)`

`[1] 1 1 1 0 0 0 1 1 1 0`

`tapply(x, v, mean)`

`0 1`

`6.25 5.00`

`tapply(x, v, median)`

`0 1`

`5.5 5.0`

compare with

`mean(x[v == 0])`

`[1] 6.25`

`mean(x[v == 1])`

`[1] 5`

Now it's your turn to try it out

Rainfall data

- Maximum rainfall at each station

`sapply(rain, max)`

- 99th percentile of rainfall at each station

`sapply(rain, quantile, probs = 0.99)`

Rainfall data

- Check that the number of recordings at each station matches the number of days recorded at the corresponding station

`all(sapply(rain, length) == sapply(day, length))`

Rainfall data

- Number of years each station is in operation

```
Year = lapply(day, floor)
```

```
Uyear = lapply(Year, unique)
```

```
OpYear = sapply(Uyear, length)
```

For any one station:

```
length(unique(floor(day[[1]] )))
```

```
sapply(day, length(unique(floor(?))) )
```

```
sapply(day, function(x) length(unique(floor(x))))
```

- Proportion of days it rained at each station

```
sapply(rain, function(x)
```

```
  sum(x > 0) / length(x))
```

Web Caching

(Skip and experiment with these data
on your own)

Web Caching



- When you use a search engine to look for a Web page, the search engine looks through its cache.
- The cache is created by regularly crawling the web and taking copies of every page that has changed since the last time it visited the site.
- It's time consuming and costly to crawl the web.
- How often do you need to crawl the web in order to keep its stock fresh for the search engine?

Web Caching

- Data: a collection of web sites were visited regularly over a period of time to see whether or not they had changed.
- Question: Given a new website, how often would you recommend as the length of time between visiting the site?

```
load(url("http://www.math.usu.edu/~symanzik/teaching/2013
_stat6910_003_fall/Files/refreshSite.rda"))
```

```
class(refreshSite2)
[1] "list"
length(refreshSite2)
[1] 10000
```

```
class(refreshSite2[[1]])
[1] "integer"
length(refreshSite2[[1]])
[1] 13
length(refreshSite2[[2]])
[1] 3
```

```
refreshSite2[1]
```

```
[[1]]
[1] 35 134 155 157 177 204 314 315 319 350 366
369 371
```

The first web site changed 13 times.

The first change was observed on the 35th visit,
the second change on the 134th visit, etc.

```
refreshSite2[c(1, 2, 4, 6)]
```

```
[[1]]
[1] 35 134 155 157 177 204 314 315 319 350 366 369 371
```

```
[[2]]
[1] 552 604 672
```

```
[[3]]
[1] 30 36 45 65 88 154 157 166 169 197 199 204 220 223 225 235 250
[18] 251 253 311 319 321 341 346 365 367 369
```

```
[[4]]
[1] 4 5 28 52 76 100 116 129 155 196 221 318 356 438 553 562 609
```

Extracting information

- We can think of these values as units of time, e.g. if the visits occurred daily then 5 is 5 days.
- Some characteristics of interest to us are:
 - The number of changes over the observed time period
 - The length of “time” between changes
 - The times at which a change was observed

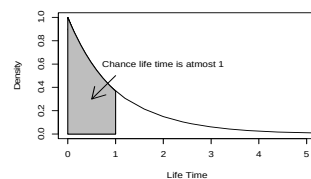
Example: One Web site

[1] 35 134 155 157 177 204 314 315 319 350
366 369 371

- The number of changes over the observed time period: 13
- The length of “time” between changes:
35, 99, 21, 2, 20, 27, 110, 1, 4, 31, 16, 3, 2
- The times at which a change was observed
These are the values given to us: 35, 134, ...

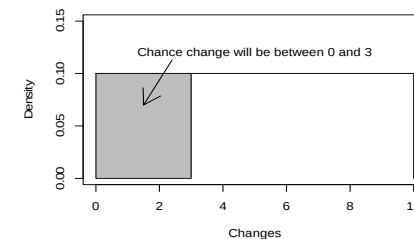
Properties of the time between

Might behave like a waiting time – the density curve for the time between changes looks like the exponential distribution



Properties of the times

- Might behave like a uniform – the density curve for the locations might be a uniform distribution (flat)



Cautions

- If the site changed more than once between visits, we wouldn't be able to tell
- We can't tell when the site changed, only that it changed at some point in the interval between visits

Matrices and Arrays

Reminder: Matrices and Arrays

- Rectangular collection of elements
- Dimensions are two, three, or more
- Homogeneous primitive elements (e.g. all numeric or all character)

- You can create a matrix in R using the `matrix` function.
- By default, matrices in R are assigned by *column-major* order.
- You can assign them by *row-major* order by setting the `byrow` argument to `TRUE`. Note that the first argument to `matrix` is a vector, so all elements must be of the same type (numeric, character, or logical).

```
m = matrix(1:6, nrow = 2)
```

```
      [,1] [,2] [,3]
[1,]  1   3   5
[2,]  2   4   6
```

Arrays – matrices in higher dimensions

```
x = array(1:30, c(4, 3, 2))
```

```
x
```

```
,, 1
```

```
  [,1] [,2] [,3]
```

```
[1,]  1  5  9
```

```
[2,]  2  6 10
```

```
[3,]  3  7 11
```

```
[4,]  4  8 12
```

```
,, 2
```

```
  [,1] [,2] [,3]
```

```
[1,] 13 17 21
```

```
[2,] 14 18 22
```

```
[3,] 15 19 23
```

```
[4,] 16 20 24
```

- The integers 1, 2, ... , 24 are arranged in a 3-dimensional array (and 25:30 are ignored)

- The array has:
 - 4 rows
 - 3 columns
 - 2 panels

```
x[1:2, 3, 2]
```

```
[1] 21 22
```

```
x[, 2, 1]
```

```
[1] 5 6 7 8
```

```
x[3:4, c(3, 1), 1]
```

```
  [,1] [,2]
```

```
[1,] 11  3
```

```
[2,] 12  4
```

apply()

- **apply(m, margin, sum)** for the matrix m, a function (here the sum) is applied. A margin of 1 applies the function to the rows, and a margin of 2 applies it to the columns.

```
m = matrix(1:6, nrow = 2)
```

```
  [,1] [,2] [,3]
```

```
[1,]  1    3    5
```

```
[2,]  2    4    6
```

```
apply(m, 1, sum)
```

```
[1] 9 12
```

```
apply(m, 2, sum)
```

```
[1] 3 7 11
```